UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos e Programação (APE)**

Aula: 05 - Entradas e Saídas com Arquivos

IA PARA ENGENHEIROS
UFRGS

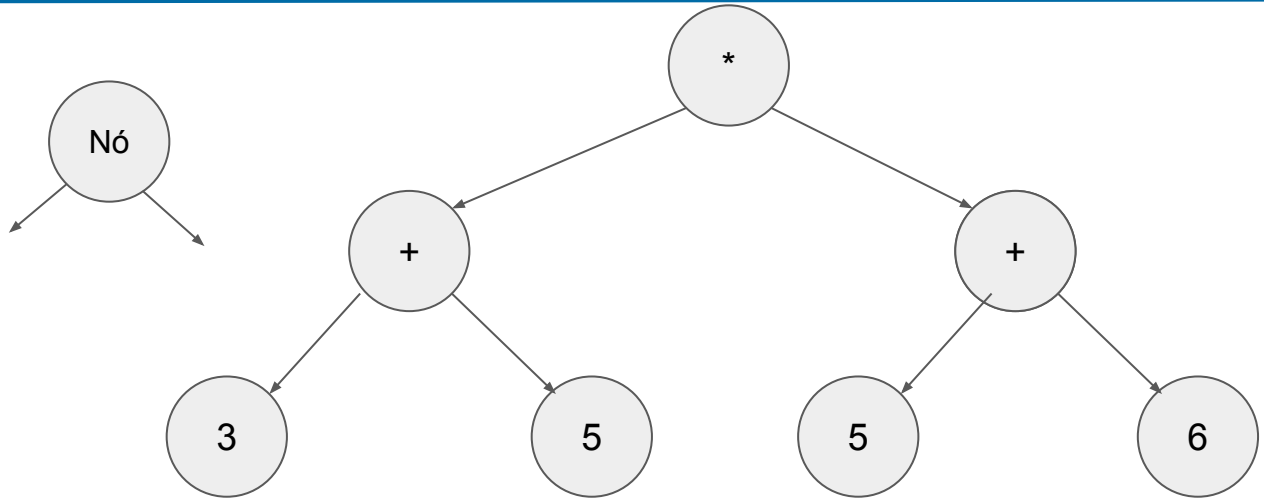
Professor: **Raphael Martins Brum**

2026

Da aula anterior… Exercício:

1) Fazer a leitura de uma “string” que representa um conjunto de operações em notação RPN.
2) Construir uma árvore (do tipo BST) que representa as operações a serem feitas.
3) Caminhar na BST para enfilerar os operandos e operadores adequadamente.
4) Utilizar uma pilha para implementar o algoritmo de cálculo baseado na notação RPN.

Exemplo de associação árvore / expressão



Caminhamento pós-fixado: 3 5 + 5 6 + *
Expressão correspondente: (3+5)*(5+6)
Resultado esperado: 8 * 11 = 88

Arquivos de texto x Arquivos binários

Arquivos de texto (plain text)

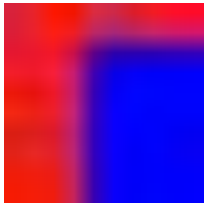
- Caracteres legíveis por humanos + caracteres especiais de estruturação (CR+LF, tab, etc.)
- Codificados segundo algum formato específico (ASCII, UTF-8)
- Úteis para facilitar a edição e compreensão dos humanos
- Exemplos: **.txt**, **.conf**, **.ini**, **.csv, …**

Arquivos binários

- Sequência de números binários que representam dados (ou mesmo instruções)
- Codificados segundo padrões consagrados ou estruturas específicas
- Exemplos: ELF, **.exe, .bmp, .dll, .jpeg, .mp3, .pdf, …**

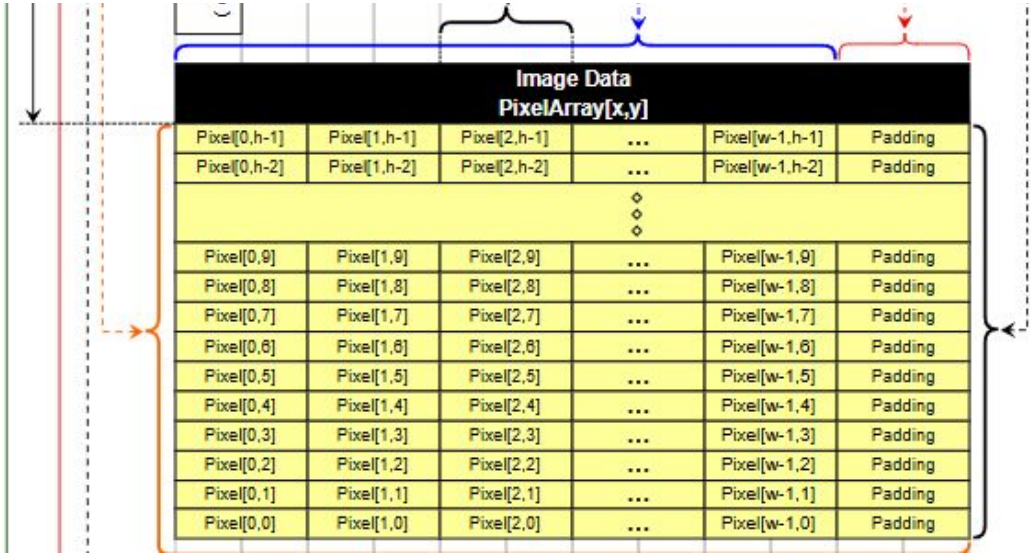
Exemplos de arquivo texto e de arquivo binário

```
apple
banana
chery
```



```
# Default settings for all hosts
Host *
    AddKeysToAgent yes
    IdentitiesOnly yes
    ServerAliveInterval 60

# Specific host shortcut
Host dev
    HostName dev.example.com
    User john
    Port 2222
    IdentityFile
~/.ssh/id_rsa_dev
```



Lendo arquivos texto

```
with open(r'c:/arquivo.txt', 'r', encoding='utf-8') as f:
    conteudo = f.read()
print(conteudo)
```

```
with open('arquivo.txt', 'r', encoding='utf-8') as f:
  for linha in f:
    print(linha)
```

```
open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None,
closefd=True, opener=None)
```

Escrevendo arquivos texto

```python
lines = ["apple\n", "banana\n", "cherry\n"]
with open("output.txt", "w") as f:
    f.writelines(lines)
```

```python
lines = ["apple", "banana", "cherry"]
with open("output.txt", "w") as f:
    for line in lines:
        f.write(f"{line}\n")
```

Formatos úteis de arquivos de texto para análise da dados

CSV (Comma-Separated Value)

- Formato de texto simples
- Valores separados por vírgulas
- Cada linha = um registro
- Primeira linha = cabeçalho (opcional)
- Ideal para dados tabulares

JSON (JavaScript Object Notation)

- Formato baseado em texto
- Estrutura de chave/valor
- Suporta listas e objetos
- Suporta dados aninhados
- Padrão para APIs web

Exemplos de arquivos CSV e JSON

CSV

```csv
nome,idade,cidade
Ana,28,São Paulo
Bruno,35,Curitiba
Carla,22,Porto Alegre
David,41,Manaus
```

JSON

```json
[
  {
    "nome": "Ana",
    "idade": 28,
    "cidade": "São Paulo"
  },
  {
    "nome": "Bruno",
    "idade": 35,
    "cidade": "Curitiba"
  }
]
```

Lendo arquivos CSV

```python
import csv

with open('dados.csv') as f:
    leitor = csv.DictReader(f)
    for linha in leitor:
        print(linha['nome'],
              linha['cidade'])
```

```python
import pandas as pd

df = pd.read_csv('dados.csv')
print(df.head())
```

`encoding='utf-8'`
Para suportar acentos e caracteres especiais

`delimiter=';'`
Especifica o tipo de seperador

`DictReader()` ou `reader()`?
DictReader retorna dicionários com os nomes das colunas

Escrevendo arquivos CSV

```python
import csv

dados = [
  ['nome', 'idade'],
  ['Ana', 28],
  ['Bruno', 35]
]

with open('saida.csv', 'w',
          newline='') as f:
    w = csv.writer(f)
    w.writerows(dados)
```


```python
import csv

dados = [
  ['nome', 'idade'],
  ['Ana', 28],
  ['Bruno', 35]
]

with open('saida.csv', 'w',
          newline='') as f:
    w = csv.writer(f)
    w.writerows(dados)
```

Lendo arquivos JSON

```python
import json

with open('dados.json') as f:
    dados = json.load(f)

print(dados[0]['nome'])
```

```python
texto = '{"nome":"Ana","idade":28}'
dados = json.loads(texto)
print(dados["nome"])  # Ana
```

JSON	Python
object { }	dict
array [ ]	list
string	str
number	int / float
true / false	True / False
null	None

Escrevendo arquivos JSON

```python
import json

dados = [
  {'nome': 'Ana', 'idade': 28},
  {'nome': 'Bruno', 'idade': 35}
]
with open('saida.json','w') as f:
    json.dump(dados, f,
              indent=2,
              ensure_ascii=False)
```

```python
texto = json.dumps(dados, indent=2)
print(texto)
```

```python
indent=2
Formata o JSON com indentação legível
```

```python
ensure_ascii=False
Mantém caracteres UTF-8 (acentos)
```

```python
sort_keys=True
Ordena as chaves alfabeticamente
```

```python
separators=(',', ':')
Compacta o JSON sem espaços
```

Projeto 6 (começar em aula)

Vamos cruzar duas bases de dados expressas em CSV para calcular o PIB per capita dos países do mundo e ordenar os países por essa métrica.

São fornecidas duas listas:

pib.csv : PIB dos Países (e territórios) do mundo
pop.csv : População dos Países (e territórios) do mundo

1) Processe os dois arquivos e crie uma estrutura em memória com indexação pelo nome do país e com todos os dados obtidos.
2) Processe essa estrutura para gerar o PIB per capita (adotando apenas a primeira fonte de cada dado)
3) Gere um CSV de saída com o nome do país e o PIB per capita correspondente, quando os dois dados existirem
4) Gere um segundo CSV com todos os países que não constam de uma das duas listas, indicando, na segunda coluna, a informação faltante.
5) Gere um JSON de saída com todos os dados apresentados de forma estruturada.

Projeto 7 (em casa)

Incremente a implementação da Árvore RPN para:

1) Ler as expressões a partir de um arquivo CSV
2) “Dumpar” a árvore para um arquivo JSON